The Goose SDK

Build on the agent, not around it

Goose is a single compiled binary — agent runtime, model access, tool execution, extensions, sessions — all included.

The SDK lets you embed it, connect to it, and extend it. Build your experience; let Goose handle the hard parts.

Why Build on the Goose SDK?

🔧 Without It

- Wire up LLM providers (auth, streaming, retries, fallbacks)
- Build session management, context windows, compaction
- Implement tool execution, sandboxing, permissions, security
- Write your own extension/plugin system
- Handle model switching, cost tracking, cancellation

Months of plumbing before you ship anything.

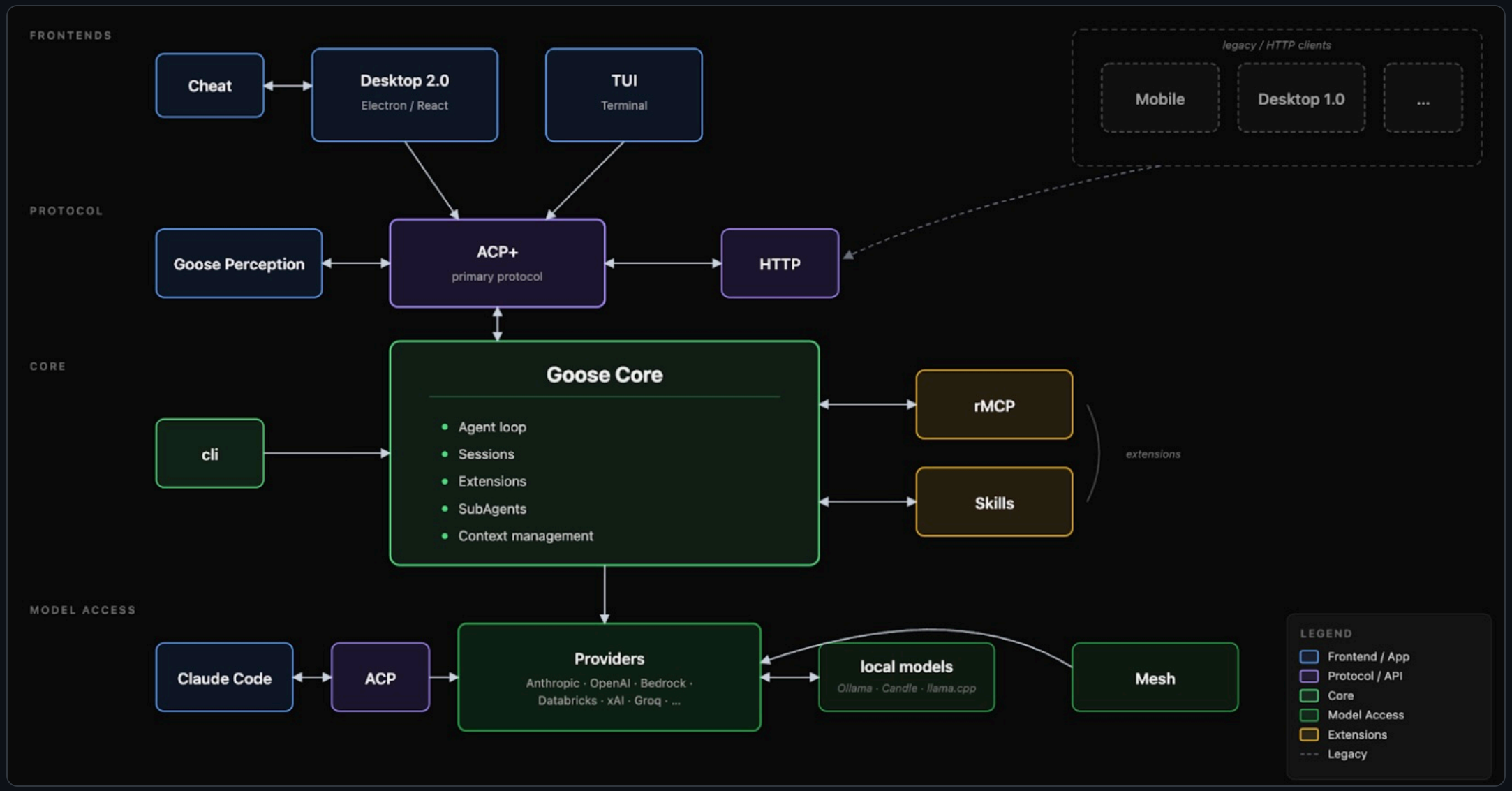
✅ With Goose SDK

- **All providers** — Anthropic, OpenAI, Bedrock, Databricks, xAI, Groq, Ollama, llama.cpp...
- **Security** — prompt injection detection, tool permissions, malware checks
- **Context management** — auto-compaction, token counting, cost tracking
- **Extensions** — MCP + rMCP, Skills, Recipes, lifecycle hooks
- **Sub-agents, scheduling, sandboxed execution**

Focus on your experience. Goose handles the rest.

One compiled binary. Brew, npm, apt, rpm, Docker, or just download it.

What's Inside the Binary



Frontends connect via ACP+ (the protocol). Everything below the protocol layer is handled for you.

How It Works

📦 Distribution

Single compiled Rust binary. Get it however you want:

- `brew install block-goose-cli`
- `npm install @aaif/goose-sdk`
- `apt` / `rpm` / `.deb` / `.flatpak`
- Docker: `ghcr.io/.../goose`
- Windows: MSI, NSIS, PowerShell script
- Curl install script, GitHub Releases
- Built-in `goose update`

🔌 Connection

Three transports, same protocol:

- **Stdio** — spawn `goose acp`, pipe JSON
- **WebSocket** — connect to running server
- **HTTP+SSE** — for web services

Runs as a daemon, child process, or CLI tool. Your app connects and sends prompts.

📡 Protocol: ACP+

JSON-RPC 2.0. The "+" is Goose's extensions beyond the ACP open standard:

- Provider/model management
- Extension hot-loading
- Config & secrets
- Session export/import

ACP+ extensions will be standardised as they prove out.

Your App → `GooseClient` (TypeScript SDK) → ACP+ (JSON-RPC) → `goose` binary → LLMs + Tools

The SDK in 30 Seconds

```
import { spawn } from "node:child_process";
import { Readable, Writable } from "node:stream";
import { ndJsonStream, PROTOCOL_VERSION } from "@agentclientprotocol/sdk";
import { GooseClient } from "@aaif/goose-sdk";
import { resolveGooseBinary } from "@aaif/goose-sdk/node";

// 1. Spawn the goose binary
const proc = spawn(resolveGooseBinary(), ["acp"], { stdio: ["pipe", "pipe", "ignore"] });
const stream = ndJsonStream(Writable.toWeb(proc.stdin!), Readable.toWeb(proc.stdout!));

// 2. Create the client
const client = new GooseClient(() => ({
  sessionUpdate: async (p) => {
    if (p.update.sessionUpdate === "agent_message_chunk" && p.update.content.type === "text")
      process.stdout.write(p.update.content.text);  // stream text to your UI
  },
  requestPermission: async (p) => ({
    outcome: { outcome: "selected", optionId: p.options[0].optionId }  // auto-approve
  }),
}), stream);

// 3. Initialize, create session, prompt
await client.initialize({ protocolVersion: PROTOCOL_VERSION, clientInfo: { name: "my-app", version: "1.0" }, clientCapabilities: {} });
```

That's it. You have a full agent with tool use, streaming, sessions, and any LLM provider.

Three Clients, One SDK

The same `GooseClient` + `@aaif/goose-sdk` powers all three:

🖥️ goose2 Desktop

Tauri + React. Connects via WebSocket.

- Singleton connection with auto-reconnect
- Zustand stores for chat state
- Replay + live streaming modes
- Full session management UI

```
const stream = createWebSocketStream(wsUrl);
const client = new GooseClient(cb, stream);
```

⌨️ Text TUI

React Ink. Connects via stdio.

- Spawns `goose acp` as child process
- Interactive TUI + one-shot text mode
- Provider onboarding flow
- ~30 lines for minimal client

```
const stream = ndJsonStream(stdin, stdout);
const client = new GooseClient(cb, stream);
```

👁️ goose_perception

Headless Node.js. Watches screen → upda

- Multi-model: fast (OCR) + smart (synth
- Sends images via ContentBlock
- Fresh sessions per task batch
- Token usage tracking

```
await client.prompt({ sessionId, prom
  { type: "text", text: "Transcribe"
  { type: "image", data: b64, mimeTyp
]});
```

What's Inside the Binary

🔒 Security

- **Prompt injection detection** — pattern + ML classifier
- **Tool permissions** — 4 modes: Auto, Approve, SmartApprove, Chat
- **Malware checks** — OSV database queries before installing extensions
- **Egress/exfiltration detection**
- **Extension sandboxing**

🧠 Context & Sessions

- **Auto-compaction** at 80% context window
- **Token counting** — tiktoken BPE, includes tool definitions
- **Cost tracking** — per-provider usage, cache-aware
- **Session persistence** — create, resume, fork, export
- **Chat recall** across sessions

🤖 Agent Capabilities

- **Sub-agents** — parallel child agents with isolated context
- **Scheduling** — background/recurring agent tasks
- **Sandboxed code execution** — Docker containers
- **Tool loop detection** — catches repetition
- **Retry with backoff**

🧩 Extensions & Skills

- **MCP + rMCP** — local and remote tool servers, hot-loadable
- **Skills** — discoverable SKILL.md files (project + global)
- **Recipes** — declarative YAML agent configs, shareable, deeplink-able
- **Lifecycle hooks** WIP — shell/HTTP at agent lifecycle points

📡 Model Access

- **Cloud** — Anthropic, OpenAI, Bedrock, Databricks, xAI, Groq...
- **Local** — Ollama, llama.cpp, Candle
- **Mesh** — distributed model routing
- **Claude Code** — as a provider via ACP
- **Observability** — OpenTelemetry + Langfuse tracing

Extension Points

Ways to extend Goose without forking it:

🧩 MCP + rMCP Extensions

Add tools at runtime via local or remote MCP servers. Goose manages the lifecycle — hot-loadable, no restart.

```
await client.extMethod(
  "_goose/extensions/add", {
    sessionId,
    name: "my-tool-server",
    uri: "http://localhost:8080",
  }
);
```

🪝 Lifecycle Hooks WIP

Shell commands or HTTP calls at key agent lifecycle points. Configured in YAML/JSON — no code changes.

- **before_tool_call** — block dangerous actions
- **after_tool_call** — audit, log, react
- **post_compact** — re-inject context after compaction
- **on_session_start / end** — setup / teardown

PRs: #8842, #7411

📚 Skills

Discoverable `SKILL.md` files with frontmatter. Project-scoped (`.goose/skills/`) or global (`~/.agents/skills/`). Procedural knowledge the agent loads on demand.

🍳 Recipes + Sub-Agents

Recipes: Declarative YAML — bundle prompt, extensions, model, parameters. Shareable, deeplink-encodable, schedulable.

Sub-agents: Spawn child agents with isolated contexts for parallel workstreams.

ACP+ → Open Standard

ACP is already an open protocol with shared infrastructure. Goose's extensions (`_goose/*`) follow a path to standardisation:

1. Ship in Goose → 2. Validate with clients → 3. Propose to spec → 4. Other agents adopt

Shared Infrastructure Today

- `@agentclientprotocol/sdk` — TypeScript SDK (npm)
- `sacp` — Rust crate (symposium-acp)
- `agent-client-protocol-schema` — shared types (crates.io)
- JSON Schema definitions for all request/response types

What Gets Standardised Next

- Provider discovery & model listing
- Extension management (add/remove/toggle)
- Config & secret management
- Session export/import
- Lifecycle hooks

Battle-tested in production across 3+ clients before proposing.

The LSP playbook: one protocol, many clients, many agents. Build for `GooseClient` today → works with any ACP agent tomorrow.

```
// Singleton connection with auto-reconnect
let client: GooseClient | null = null;

export async function getClient() {
  if (client) return client;

  const wsUrl = await invoke("get_goose_serve_url");
  const stream = createWebSocketStream(wsUrl);

  client = new GooseClient(() => ({
    sessionUpdate: async (n) => {
      notificationHandler.handle(n);
    },
    requestPermission: async (p) => ({
      outcome: { outcome: "selected",
        optionId: p.options[0].optionId },
    }),
  }), stream);

  await client.initialize({
    protocolVersion: PROTOCOL_VERSION,
    clientInfo: { name: "goose2", version: "0.1.0" },
    clientCapabilities: {},
  });

  // Auto-reconnect on disconnect
  client.closed.then(() => { client = null; });
  return client;
}

// Clean API wrappers
export const prompt = (sid, blocks) =>
  getClient().then(c =>
    c.prompt({ sessionId: sid, prompt: blocks }));

export const setModel = (sid, model) =>
  getClient().then(c =>
```

- **Lazy connection** — connects on first use
- **Auto-reconnect** — nulls client on WS close, next call reconnects
- **All state in Zustand** — notifications route to stores
- **Session replay** — `loadSession()` replays history via notifications

Notification → UI

```
// Route to Zustand stores
switch (update.sessionUpdate) {
  case "agent_message_chunk":
    chatStore.append(sid, update.content);
    break;
  case "tool_call":
    chatStore.addTool(sid, update);
    break;
  case "usage_update":
    chatStore.updateUsage(sid, update);
    break;
}
```

Pattern: Headless Service (goose_perception)

```
// Multi-model: cheap for OCR, expensive for synthesis
async function createSession(kind: "fast"|"smart") {
  const provider = kind === "fast"
    ? config.fastProvider : config.smartProvider;
  const model = kind === "fast"
    ? config.fastModel : config.smartModel;

  const s = await client.newSession({
    cwd: wikiDir, mcpServers: [],
    _meta: { provider },
  });

  if (model) await client.goose
    .GooseSessionProviderUpdate({
      sessionId: s.sessionId, provider, model,
    });

  return s.sessionId;
}

// Send screenshot for transcription
const sid = await createSession("fast");
buf.length = 0;
await client.prompt({ sessionId: sid, prompt: [
  { type: "text", text: "Transcribe this screen" },
  { type: "image", data: screenshot.base64,
    mimeType: "image/png" },
]});
const result = buf.join("");
```

Key Patterns

- **Session-per-task** — fresh session for each batch, no context buildup
- **Multi-model** — different sessions with different providers/models
- **Stream buffer** — accumulate chunks, read after prompt completes
- **Images as ContentBlock** — base64 alongside text

Query Providers at Runtime

```
// What providers are configured?
const providers = await client.goose
  .GooseProvidersDetails({});
providers.providers
  .filter(p => p.isConfigured)
  .forEach(p =>
    console.log(`${p.name}: ${p.displayName}`));

// What models does a provider have?
const models = await client.goose
  .GooseProvidersModels({
    providerName: "anthropic"
  });
```

Build on Goose

📦 One Binary, Runs Anywhere

Compiled Rust. Brew, npm, apt, rpm, Docker, Windows MSI, curl. Runs as daemon, child process, or CLI.

🔒 Security Built In

Prompt injection detection, tool permission modes, malware checks, egress monitoring, sandboxed execution.

🧩 Extend, Don't Fork

MCP + rMCP for tools. Skills for knowledge. Recipes for config. Hooks for policy. Sub-agents for parallelism.

📡 Open Protocol

ACP is the standard. Goose's ACP+ extensions prove out in production, then get standardised. Build once, work with any ACP agent.

3 clients prove it works. Your app could be the 4th.

goose2 Desktop | **Text TUI** Terminal | **goose_perception** Headless

```
brew install block-goose-cli    # or npm, apt, docker, curl...
```